

UNIFACS — Universidade Salvador
UC Sistemas Distribuídos e Mobile

Relatório — Trabalho A3

Sistema distribuído de captação de dados de venda — Rede de Drogarias

Equipe:

Wadson Daniel
André Filipe
Lucas Lopes

1. Identificação da equipe

Nome completo	Matrícula
Wadson Daniel	1272216683
André Filipe	1272212034
Lucas Lopes Pereira Pedroza	1272215973

Repositório (tag EntregaA3): https://github.com/Wadls/A3_Sistemas_Distribuidos

2. Visão geral do projeto

A aplicação simula a captação de dados de venda de uma rede de lojas, no contexto de uma rede de drogarias. O sistema gerencia clientes, vendedores e estoque de produtos, processa pedidos de venda e cancelamentos, e gera relatórios estatísticos sobre vendas e estoque.

A solução é distribuída: foi dividida em duas aplicações independentes — a API Principal, que concentra os cadastros e o processamento de vendas, e a API de Relatórios, dedicada exclusivamente à geração dos relatórios estatísticos. As duas executam em containers Docker, junto a um banco de dados relacional MySQL, orquestrados por Docker Compose.

O banco é inicializado com uma carga mínima de dados (5 clientes, 2 vendedores, 10 produtos e 15 vendas de exemplo), atendendo ao requisito de população inicial.

3. Requisitos de software

3.1 Linguagem e runtime

- **Linguagem:** JavaScript (requisito obrigatório do trabalho).
- **Ambiente de execução:** Node.js 18 (imagem node:18-alpine usada nos containers).

3.2 Bibliotecas e dependências

Ambos os serviços usam as mesmas bibliotecas principais (conforme os respectivos package.json):

Dependência	Versão	Função
express	^5.2.1	Framework web; define rotas e middlewares das APIs REST.
mysql	^2.18.1	Driver de conexão com o banco de dados MySQL.

3.3 Banco de dados

- **SGBD:** MySQL 8.0 (banco de dados relacional, conforme exigido).
- **Base:** drogaria, criada e populada na inicialização a partir de infraestrutura/script_banco.sql.

3.4 Containerização

- **Docker e Docker Compose** para empacotar e orquestrar os três serviços (API Principal, API de Relatórios e MySQL).
- **Imagens base:** node:18-alpine para as APIs; mysql:8.0 para o banco.

4. Instruções de instalação e execução

4.1 Pré-requisitos

- Git
- Docker e Docker Compose

4.2 Passo a passo

Com o Docker em execução, basta clonar o repositório e subir todo o ambiente com um único comando:

```
# 1. Clonar o repositório
git clone [link do repositório]

# 2. Entrar na pasta do código-fonte (onde está o docker-compose.yml)
cd [nome-do-repositorio]/codigo_fonte

# 3. Construir as imagens e subir os três containers
docker compose up --build
```

Para encerrar, use Ctrl+C e depois docker compose down (use docker compose down -v para apagar também os dados do banco).

4.3 Serviços e portas

Serviço	Porta	URL base
API Principal	3000	http://localhost:3000
API de Relatórios	3001	http://localhost:3001
Banco de dados (MySQL)	3306	localhost:3306

4.4 Endpoints da API Principal (porta 3000)

Rotas genéricas de CRUD, em que :tabela pode ser cliente, vendedor ou estoque:

Método	Rota	Ação
GET	/:tabela	Lista todos os registros.
GET	/:tabela/:id	Busca um registro por ID.
POST	/:tabela	Cria um registro (corpo em JSON).
PUT	/:tabela/:id	Atualiza um registro por ID.
DELETE	/:tabela/:id	Remove um registro por ID.

Rotas específicas de venda (CRUD direto em /vendas é bloqueado de propósito):

Método	Rota	Ação
POST	/vendas/pedido	Recebe um pedido: valida e baixa o estoque de forma transacional.
PUT	/vendas/:id/cancelar	Cancela um pedido e devolve a quantidade ao estoque.

4.5 Endpoints da API de Relatórios (porta 3001)

Método	Rota / Relatório
GET	/relatorios/produtos-mais-vendidos
GET	/relatorios/produtos-por-cliente
GET	/relatorios/consumo-medio
GET	/relatorios/baixo-estoque?limite=80 (limite opcional; padrão 50)

5. Arquitetura, estratégias e algoritmos (item d do enunciado)

5.1 Padrão de arquitetura: Microsserviços

O sistema adota uma arquitetura de Microsserviços, atendendo ao requisito de que o serviço de geração de relatório execute em uma aplicação distinta do restante do sistema. São duas aplicações independentes, cada uma com seu próprio container, código e porta:

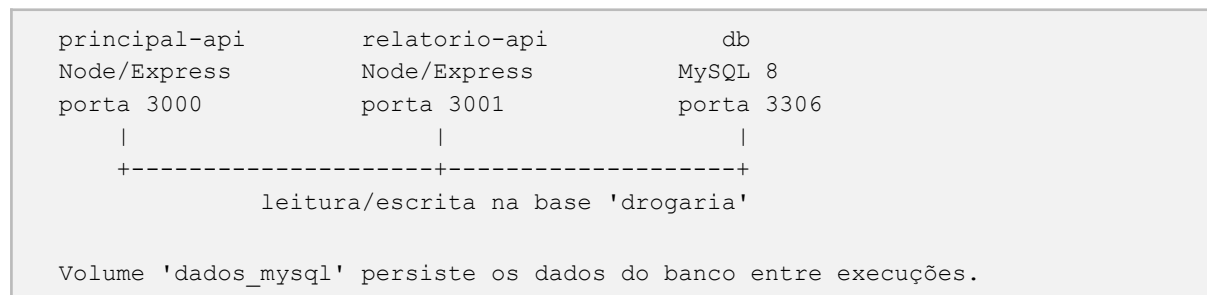
- **API Principal (porta 3000):** CRUD de cliente, vendedor e estoque, e o processamento de vendas (receber e cancelar pedido).
- **API de Relatórios (porta 3001):** geração dos quatro relatórios estatísticos, em modo somente leitura.

As duas aplicações compartilham o mesmo banco de dados relacional (MySQL). A API de Relatórios lê diretamente da base drogaria para produzir as estatísticas. Essa decisão foi tomada por simplicidade e porque os relatórios são consultas de leitura/agregação; a contrapartida é o acoplamento das duas aplicações ao mesmo esquema de banco.

Internamente, cada serviço segue o padrão MVC em camadas:

```
Requisição HTTP
|
[Router]      -> routers/      (define rotas, repassa ao controller)
|
[Controller]  -> controllers/   (orquestra a operação)
|
[Model]       -> models/       (acesso a dados / SQL)
|
[Infraestrutura] -> infraestrutura/ (conexão e carga inicial do banco)
|
Banco MySQL
```

Visão dos containers orquestrados pelo Docker Compose (rede rede-drogaria):



5.2 Modelo de dados (banco relacional)

Definido em infraestrutura/script_banco.sql:

- **cliente** (id_cliente PK, nome, data_nascimento, email único)
- **vendedor** (id_vendedor PK, nome, data_nascimento, email único, farmacia)
- **estoque** (id_produto PK, nome_produto único, quantidade)
- **vendas** (id_venda PK, id_cliente FK, id_produto FK, qtd, status ∈ {Confirmado, Cancelada}, id_vendedor FK)

A tabela vendas relaciona-se com cliente, estoque e vendedor por chaves estrangeiras, garantindo integridade referencial. O campo status é restringido por uma checagem (CHECK) aos valores Confirmado e Cancelada.

5.3 Padrões de projeto

O enunciado exige no mínimo 4 padrões de projeto na implementação (distintos do padrão de arquitetura). Foram aplicados os quatro padrões a seguir, todos presentes no código:

1. Singleton (GoF, criacional)

Os controllers, models e a infraestrutura são exportados como uma única instância compartilhada (module.exports = new Classe()), reutilizada em toda a aplicação.

Onde: object_Controller.js, object_Models.js, tabelas.js (API Principal);
relatorioController.js, relatorioModel.js (API de Relatórios).

2. DAO (Data Access Object) / Data Mapper (acesso a dados)

As classes de model concentram todo o acesso ao banco (montagem de SQL e execução), isolando os controllers da persistência. Os controllers nunca escrevem SQL.

Onde: models/object_Models.js e models/relatorioModel.js.

3. Facade (GoF, estrutural)

Os controllers atuam como uma fachada (Facade): oferecem uma interface única e simplificada para o subsistema de acesso a dados. A camada de rotas interage apenas com métodos diretos do controller (buscar, criar, atualizar, deletar, criarPedidoVenda, cancelarPedidoVenda) e não precisa conhecer a complexidade interna dos models — a montagem de SQL, as transações, as validações e o tratamento de erros ficam encapsulados atrás dessa interface.

Onde: controllers/object_Controller.js (AtendimentoController) e
controllers/relatorioController.js (RelatorioController).

4. Front Controller (ponto de entrada único)

Um conjunto único de rotas genéricas (/:tabela) recebe e despacha as requisições de todas as entidades a partir de um controller genérico, em vez de um controller por entidade.

Onde: routers/object_Router.js + controllers/object_Controller.js.

5.4 Estratégias e algoritmos

CRUD genérico. Em vez de duplicar lógica por entidade, o controller e o model recebem o nome da tabela como parâmetro e derivam dinamicamente a chave primária correspondente (mapa cliente → id_cliente, vendedor → id_vendedor, etc.), centralizando o acesso a dados.

Processamento de vendas (transacional). Ambas as operações usam transação com trava de linha (SELECT ... FOR UPDATE) para evitar condições de corrida, com rollback em qualquer falha:

- **Receber pedido:** verifica a existência do produto, valida se há estoque suficiente, baixa a quantidade do estoque e registra a venda com status Confirmado.
- **Cancelar pedido:** verifica se a venda existe e não está cancelada, devolve a quantidade ao estoque e altera o status para Cancelada (a venda permanece no histórico).

Relatórios estatísticos. Cada relatório é uma consulta SQL de agregação sobre as vendas confirmadas:

Relatório	Lógica (SQL)
Produtos mais vendidos	SUM(qtd) por produto (status Confirmado), ordenado do mais vendido ao menos vendido.
Produtos por cliente	JOIN de vendas com cliente e estoque; SUM(qtd) agrupado por cliente e produto.
Consumo médio do cliente	AVG(qtd) e SUM(qtd) por cliente, mostrando média de itens por pedido e total comprado.
Produto com baixo estoque	Produtos com quantidade <= limite (parâmetro ?limite na URL; padrão 50), ordenados do menor estoque.

6. Status das funcionalidades (requisito → situação)

Requisito do enunciado	Situação
CRUD cliente / vendedor / estoque	✓ Implementado
Receber pedido de compra	✓ Implementado (transacional)
Cancelar pedido de compra	✓ Implementado (transacional)
Relatório: produtos mais vendidos	✓ Implementado
Relatório: produto por cliente	✓ Implementado
Relatório: consumo médio do cliente	✓ Implementado
Relatório: produto com baixo estoque	✓ Implementado
Carga inicial (≥10 produtos, 5 clientes, 2 vendedores)	✓ Implementado
Aplicação em JavaScript	✓ Atendido
Banco relacional	✓ Atendido (MySQL 8)
Execução em containers Docker	✓ Implementado (Compose, 3 serviços)
Serviço de relatório em aplicação distinta	✓ Implementado (API separada, porta 3001)
Padrão de arquitetura definido	✓ Microserviços + MVC
≥ 4 padrões de projeto	✓ 4 implementados

7. Considerações finais

O desenvolvimento deste trabalho permitiu colocar em prática os principais conceitos da disciplina de Sistemas Distribuídos. Dividir a aplicação em dois serviços independentes — um para as vendas e os cadastros, e outro só para os relatórios — deixou clara a ideia de microserviços e nos obrigou a tomar decisões de projeto, e não apenas escrever código. Nosso maior desafio foi fazer as três partes do sistema (as duas APIs e o banco) subirem juntas e na ordem certa dentro do Docker, além de garantir que o estoque ficasse sempre correto a cada venda ou cancelamento.

No fim, o principal aprendizado da equipe foi perceber que a maior dificuldade de um sistema distribuído não está em uma funcionalidade isolada, mas em fazer as partes conversarem bem e se manterem organizadas — algo que a arquitetura em camadas e os padrões de projeto nos ajudaram a alcançar.